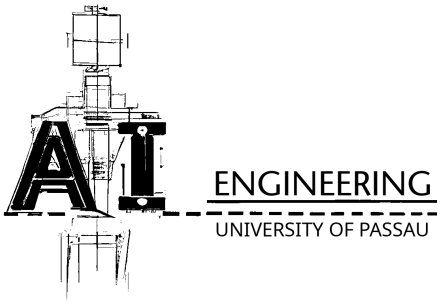
Mining Software Repositories

Exercise 4

Prof. Dr. Steffen Herbold
Winter 2026
Due on 11.11.2026



Author: Lukas Schulte

Task 1: Expanding the harmonized database

1.1 Expanding the number of projects

In week's tasks, we will expand the number of projects from which data needs to be collected. Familiarize yourself with the following repositories and find out how they handle their commits, issues and pull requests (PRs):

- Apache Zookeeper
- Apache Superset
- At least one more project, which you choose by yourself. Check section 4.1 for support and make sure that you can explain your choice, including how it handles commits, issues, etc.

1.2 Revisiting exercise 3

In exercise 3 you have designed a database schema and populated it with data from the Apache Ignite project. Now you are going to extend it with data from the new projects.

- Revisit your database schema. Does the data from the new projects fit in there as well, or do you need to make adjustments?
- Mine the data from the new projects and store it in the database. Make sure that although tables may now contain data from multiple projects, you can still trace each row back to its source.

You should first test the import on a subset of the data and then let the full import run overnight. It might take multiple hours.

Task 2: Linking

We can also create links between issues and commits, as well as issues and PRs. For this, we need to remember the patterns we already spotted in our data.

- Come up with a way to link issues to commits, commits to PRs, and PRs to issues. You will require some knowledge about regular expressions (RegEx). More information on RegEx can be found in section 4.2.

- Implement your idea and store the required information in your database (foreign keys, primary keys, etc.). Once you are finished, it should be possible to create a SQL query that, for example, returns all commits related to an issue based on the issue's id.

Task 3: Metrics

3.1 Calculating basic metrics

Use the data stored in your database to calculate the following metrics. Calculate those metrics per project as well as over all projects in your database.

- Mean time to resolve: the mean of the time that it takes for an issue to be resolved. Differentiate between different types of issues.
- Mean time to response: the mean of the time that it takes for someone, other than the creator, to respond to an PR.

How would you classify those metrics according to the categories discussed in the lecture?

3.2 Calculating metrics based on linked data

Thanks to the linking done in Task 2, we can also calculate more advanced metrics. Use the linked information to calculate the number of bugs for each file.

1. Follow the relation of bugs (issues with type bug) to the connected commits and extract the files that changed.
2. Count the number of occurrences of each file. Which files are connected to the highest number of bugs? Which files are barely ever connected to bugs? Is there a pattern?

How would you classify this metric according to the categories discussed in the lecture?

Recommended Tools

4.1 Seart GitHub Search

The Seart GitHub Search Tool can help you find a sufficiently large GitHub repository for this task.[1]

4.2 RegEx

RegEx is a textual pattern matching technique. You can find a tutorial and a short python reference online.[2][3] Further, you may want to try out your patterns in a testing environment like the RegExr tool.[4]

[1] Seart GitHub Search: `https://seart-ghs.si.usi.ch`
[2] RegEx for python: `https://regexone.com/references/python`
[3] RegEx tutorial: `https://regexone.com`
[4] RegExr tool: `https://regexr.com`